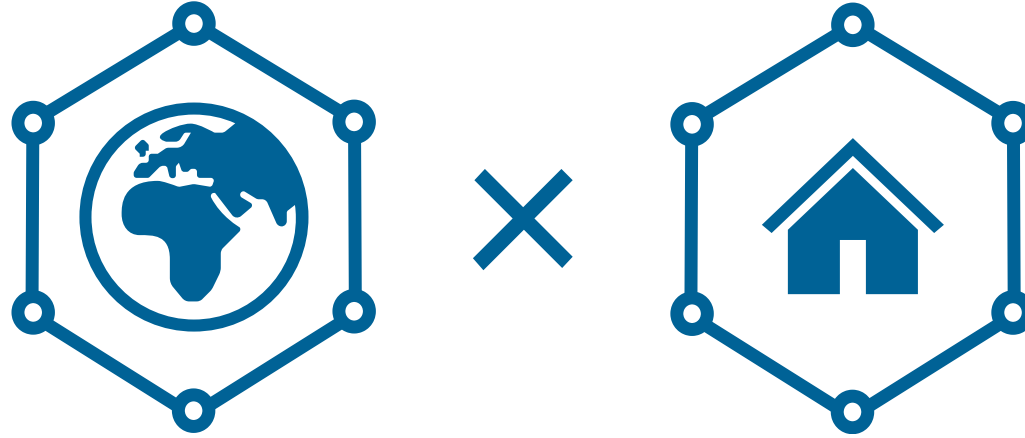




Applications and Architectures
Responsibility Driven
KG Architecture Design

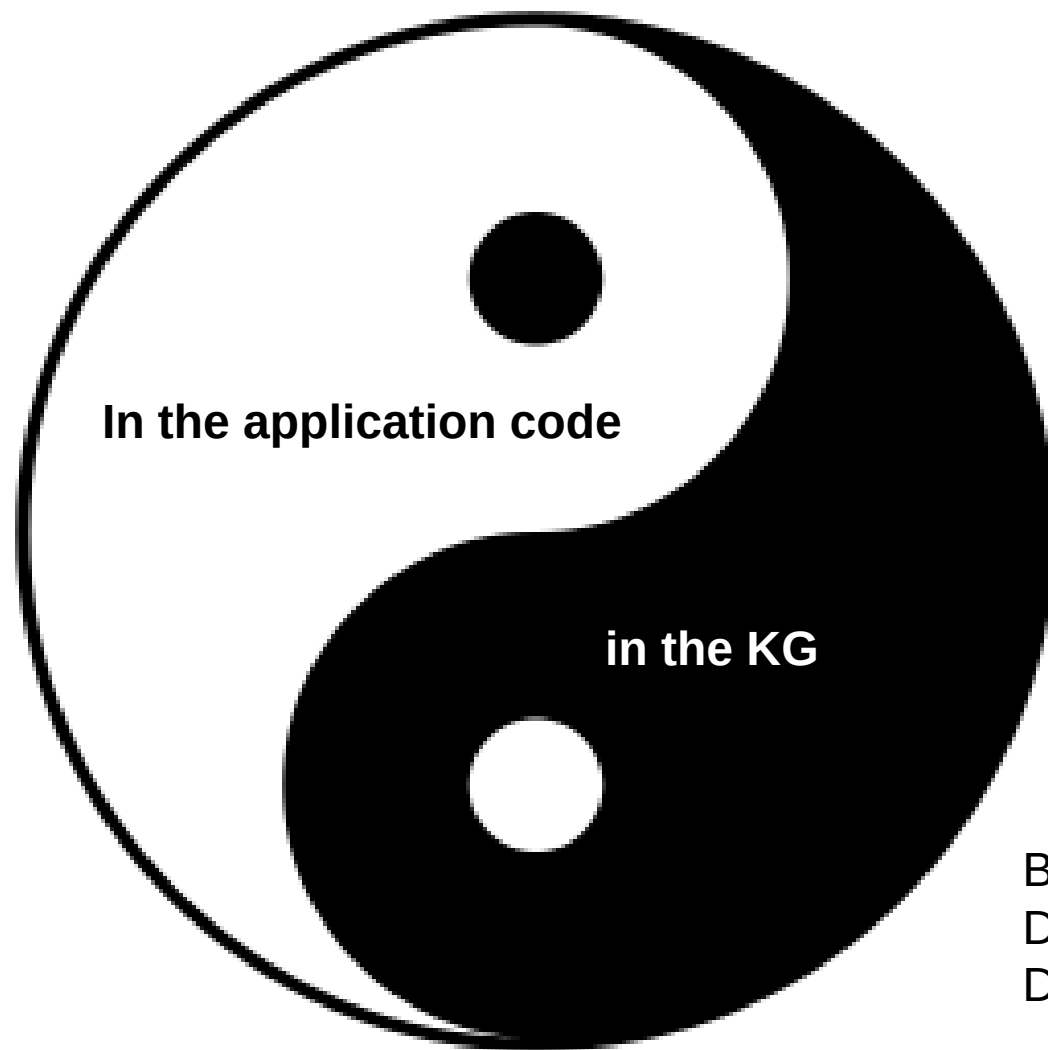
Emanuel Sallinger



Application Code vs External Reasoning

Where is the application logic in KG-based applications?

Front-end over back-end
Sophisticated visualisations
Data-independent process
Data-independent algorithms



Back-end over front-end
Data-driven process
Data-dependent algorithms



Responsibility Driven Design

Responsibility of knowing

who has responsibility to know x between *application logic* and *knowledge graph*?

- **Assign**, extensionally known, original information to the ground extensional component of the KG.
- **Design** the respective schemas by adopting the conceptual and logical design frameworks that are best suited for the data model and the respective technology.
- **Map** the instances as relational constructs in the KG.
- **Model** business rules, recurring patterns, intensionally known and inferable information as inference rules of the intensional component.



Responsibility Driven Design

Responsibility of knowing

who has responsibility to know x between *application logic* and *knowledge graph*?

- **Similar to OO applications, where we design which components know what**
 - Information Expert pattern
 - Low coupling
 - High Cohesion
- **What is in the application and what is in the database?**
 - Design database first
 - Design application first
 - Meet in the middle



Responsibility Driven Design

Responsibility of doing

who has responsibility to implement behaviour x between *application logic* and *knowledge graph*?

Represent the behaviour of a software component as inference rules of a KG and make the business logic interpret it, only when it is **highly data dependent** and its specification requires **nontrivial reasoning over data**.



Responsibility Driven Design

Responsibility of doing

who has responsibility to implement behaviour x between *application logic* and *knowledge graph*?

- The functional behaviour of a KG-based application should be in the **application itself**
 - code clarity, simplicity, understandability
- **code in KG** sustains
 - modifiability, reusability
- A behaviour that depends on a **non-trivial interpretation of data** is a good candidate for being encoded as inference rules.



Responsibility Driven Design

x controls y if
x directly holds over 50% of y, or
x controls a set S of companies that jointly control y

```
own(x,y,w) , w>0.5 → control(x,y) ,  
control(x,y) , own(y,z,w) , v=msum(w,<y>) , v>0.5 → control(x,z) .
```

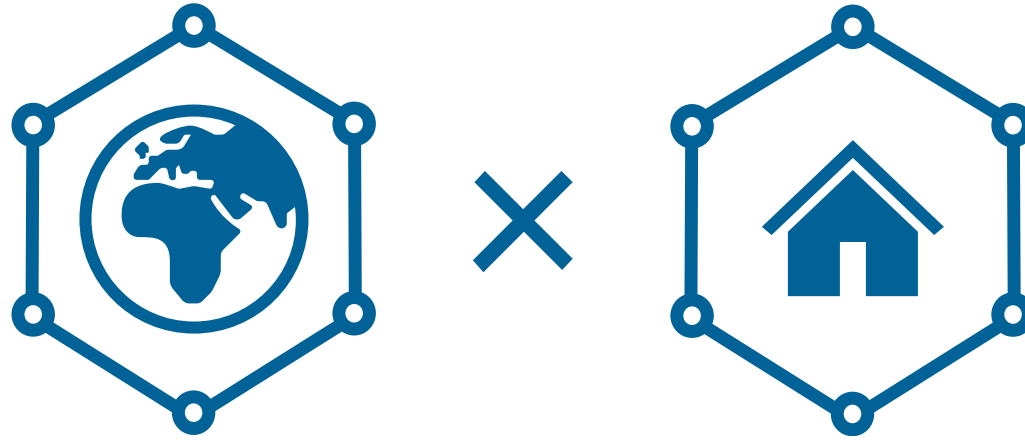
In the KG!



Responsibility Driven Design

- classify companies by region depending on their address
- calculate a risk indicator on the basis of quantitative features for each company and a given econometric model

In the application logic!
(except...)



Applications and Architectures
Responsibility Driven
KG Architecture Design

Emanuel Sallinger